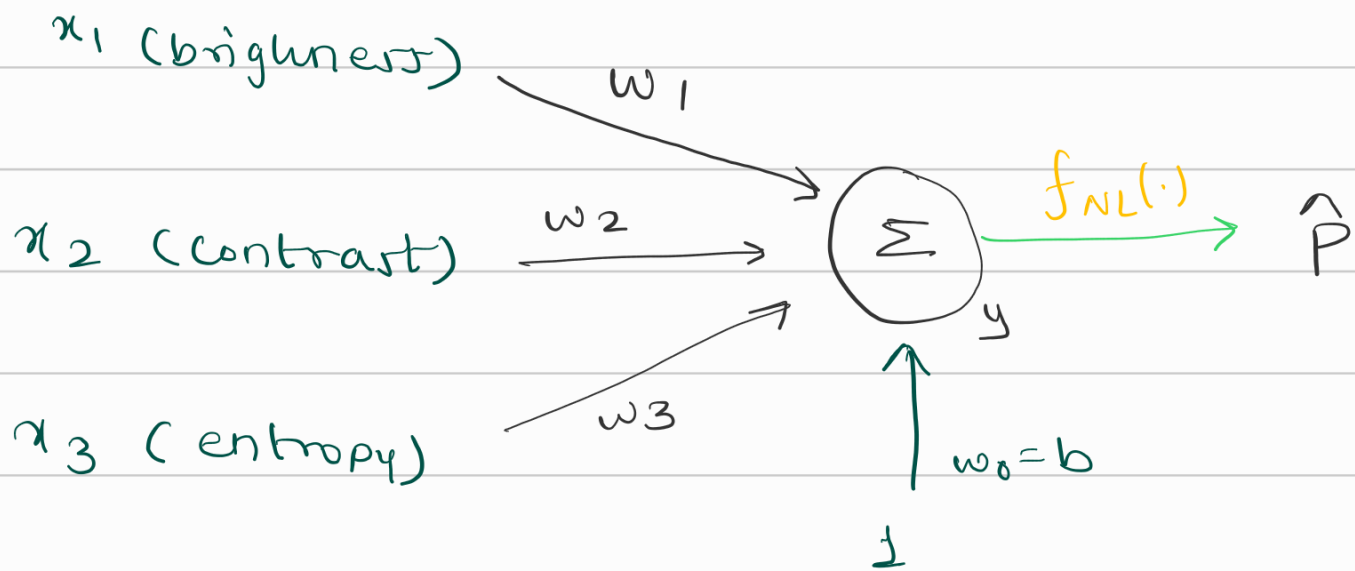


# # Simple Neuron Model



$$y = w_0 + w_1 x_1 + w_2 x_2 + w_3 x_3$$

$$y = [w \ b] [x \ 1]^T$$

$$\hat{P} = f_{NL}(y) \quad \dots \rightarrow \text{Non linear function}$$

$$f_{NL}(y) = \frac{1}{1 + e^{-y}} \quad \dots \text{sigmoid fn}$$

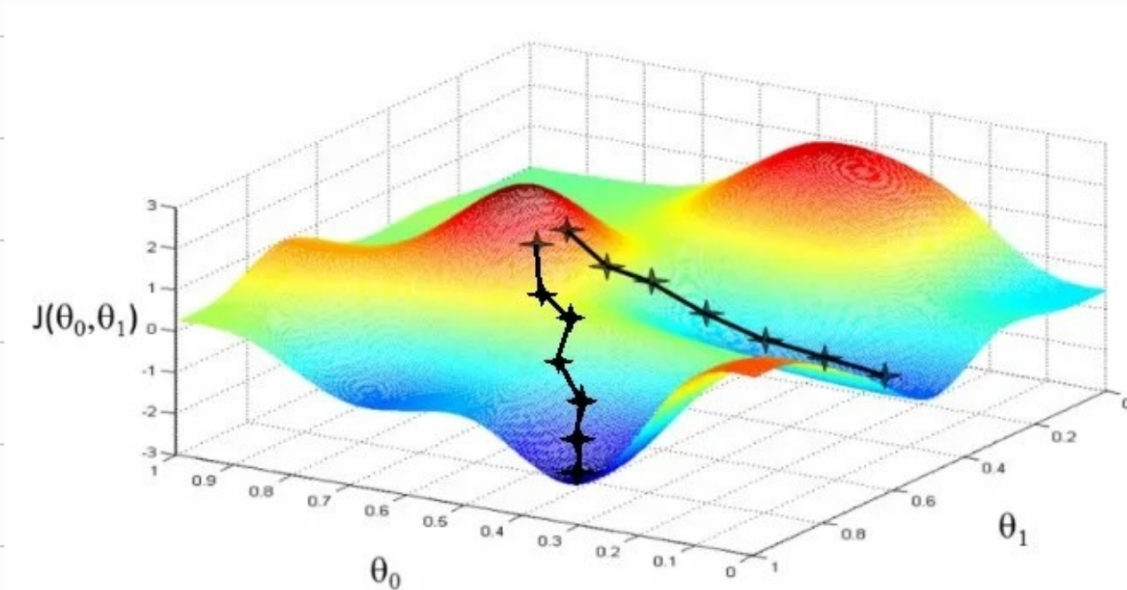
$$f_{NL}(y) = \tanh(y) = \frac{\exp(y) - \exp(-y)}{\exp(y) + \exp(-y)}$$

# # Neural Network Formulation

$$x_j \xrightarrow{w_{k,j}} \left( \sum_{j_k} \right) \xrightarrow{f_{NL}(\cdot)} \hat{p}_k$$

# # Gradient Decent

$$w^{(k+1)} = w^{(k)} - \frac{\partial}{\partial w^{(k)}} J(w)$$





# Neural Networks for Visual Computing



**Dr. Debdoot Sheet**

Assistant Professor, Department of Electrical Engineering  
Principal Investigator, Kharagpur Learning, Imaging and  
Visualization Group  
Indian Institute of Technology Kharagpur

[www.facweb.iitkgp.ernet.in/~debdoot/](http://www.facweb.iitkgp.ernet.in/~debdoot/)

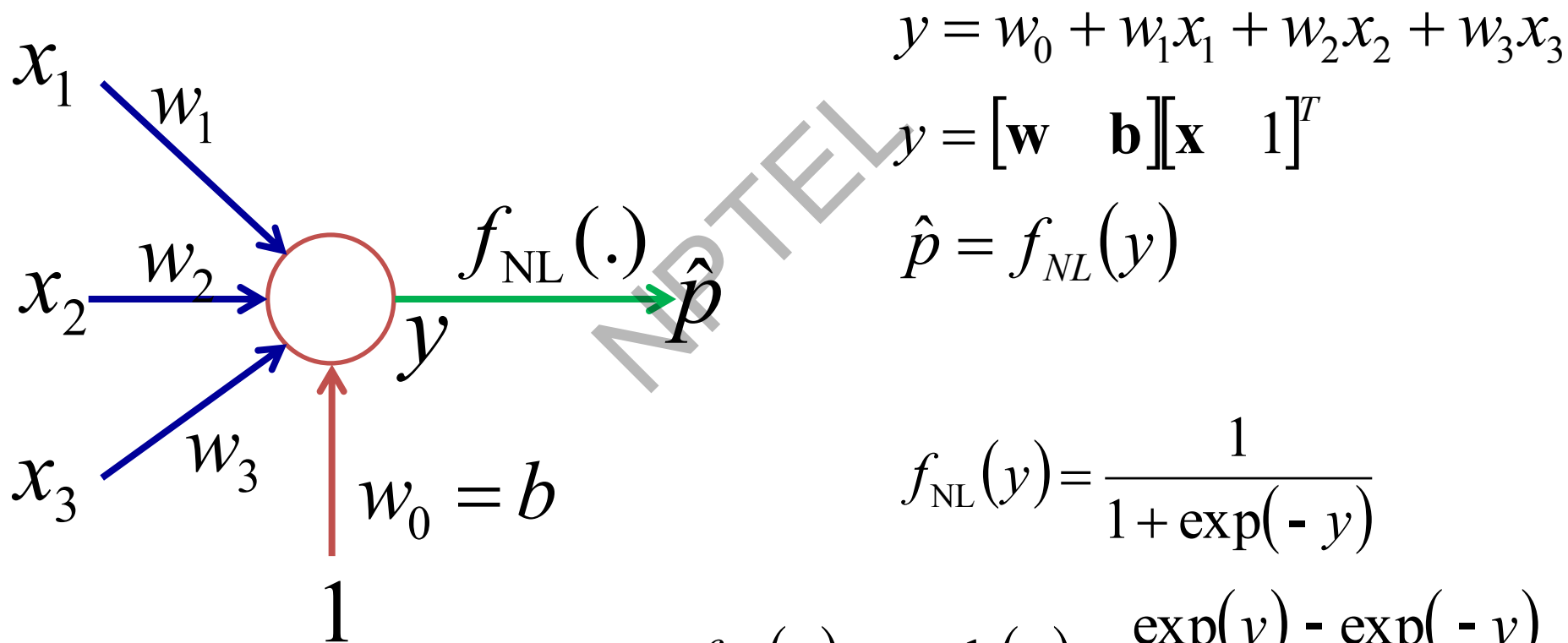


# Contents

- Simple neuron
- Neural network formulation
- Learning with error backpropagation
- Gradient checking and optimization



# Simple Neuron Model



$$y = w_0 + w_1x_1 + w_2x_2 + w_3x_3$$

$$y = [\mathbf{w} \quad \mathbf{b}] [\mathbf{x} \quad 1]^T$$

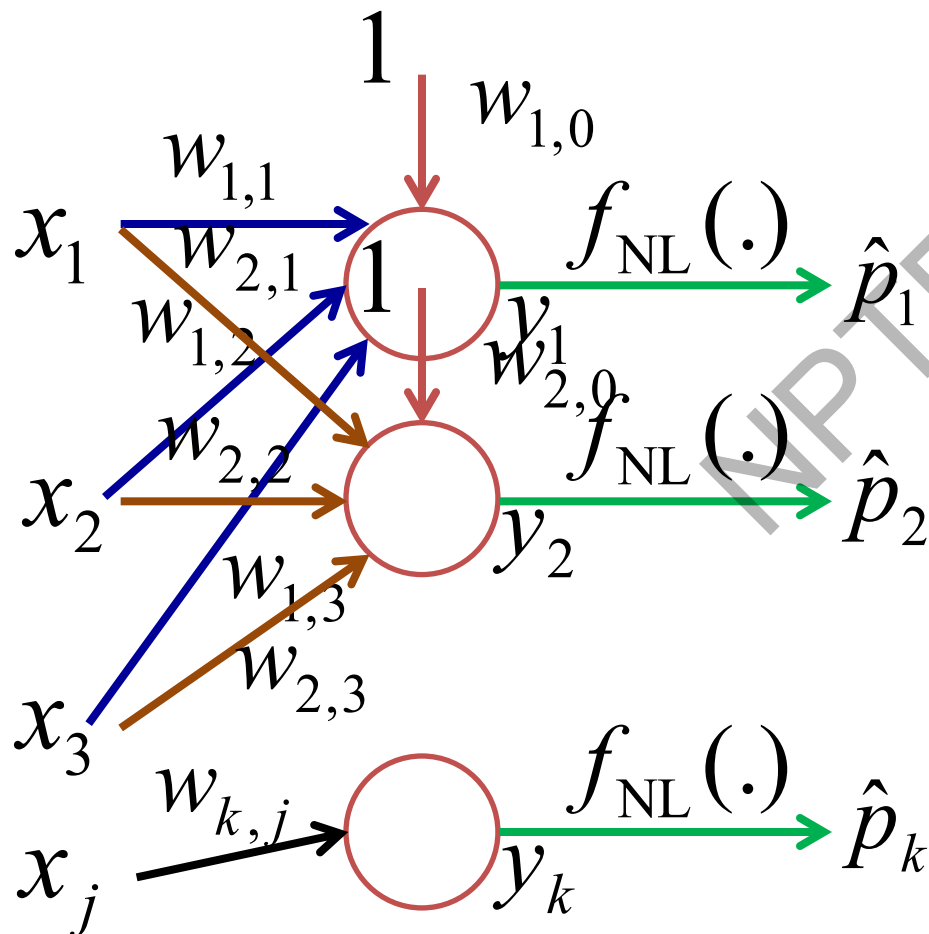
$$\hat{p} = f_{NL}(y)$$

$$f_{NL}(y) = \frac{1}{1 + \exp(-y)}$$

$$f_{NL}(y) = \tanh(y) = \frac{\exp(y) - \exp(-y)}{\exp(y) + \exp(-y)}$$



# Neural Network Formulation



$$y_1 = [\mathbf{w} \quad \mathbf{b}] [\mathbf{x} \quad 1]^T$$

$$\hat{p}_1 = f_{NL}(y_1)$$

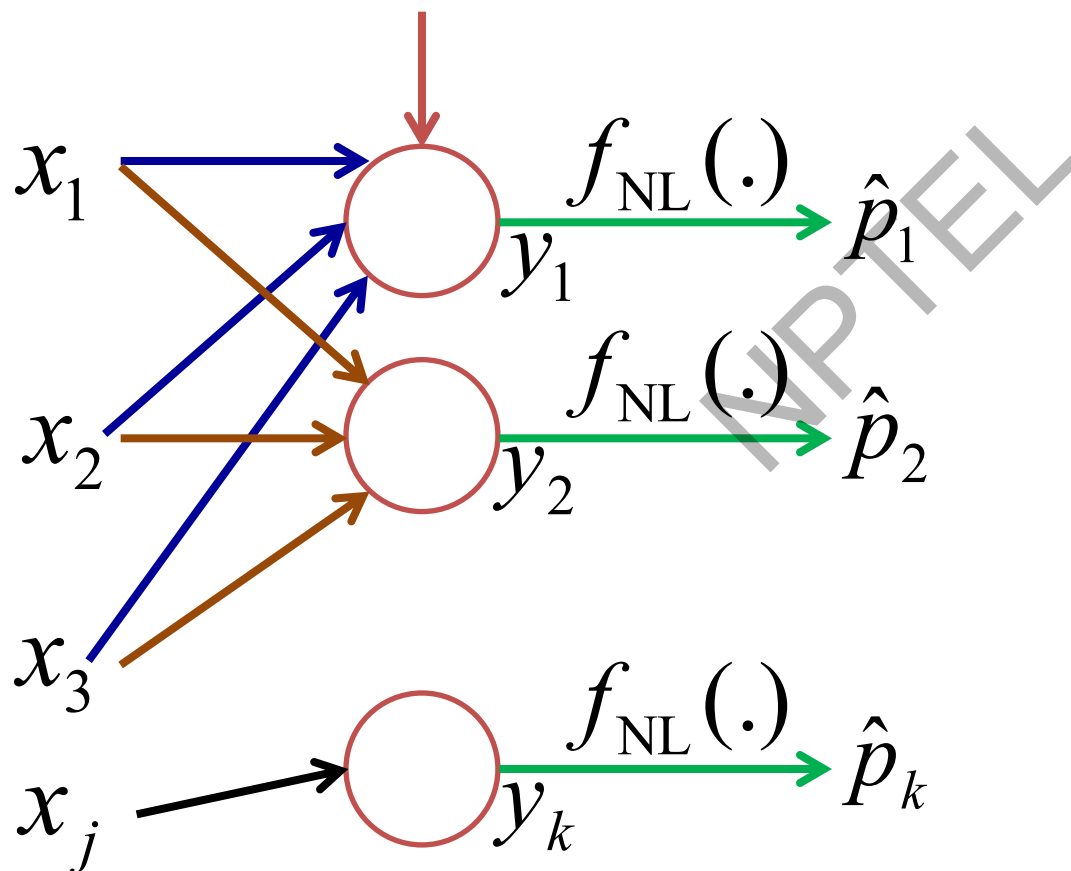
$$y_2 = [\mathbf{w} \quad \mathbf{b}] [\mathbf{x} \quad 1]^T$$

$$\hat{p}_2 = f_{NL}(y_2)$$

$$\mathbf{y} = [\mathbf{w} \quad \mathbf{b}] [\mathbf{x} \quad 1]^T$$

$$\hat{\mathbf{p}} = f_{NL}(\mathbf{y})$$

# Error in Prediction



$$e_1 = |p_1 - \hat{p}_1|$$

$$e_2 = |p_2 - \hat{p}_2|$$

$$E = \|\mathbf{p} - \hat{\mathbf{p}}\|$$

we have  
taken down  
error to  
zero



# Error Backpropagation

|                |                |                      |
|----------------|----------------|----------------------|
| $\mathbf{x}_1$ | $\mathbf{p}_1$ | $\hat{\mathbf{p}}_1$ |
| $\mathbf{x}_2$ | $\mathbf{p}_2$ | $\hat{\mathbf{p}}_2$ |
| $\mathbf{x}_3$ | $\mathbf{p}_3$ | $\hat{\mathbf{p}}_3$ |
| $\vdots$       | $\vdots$       | $\vdots$             |
| $\mathbf{x}_n$ | $\mathbf{p}_n$ | $\hat{\mathbf{p}}_n$ |

$$J(\mathbf{W}) = \sum_n \|\mathbf{p}_n - \hat{\mathbf{p}}_n\|$$

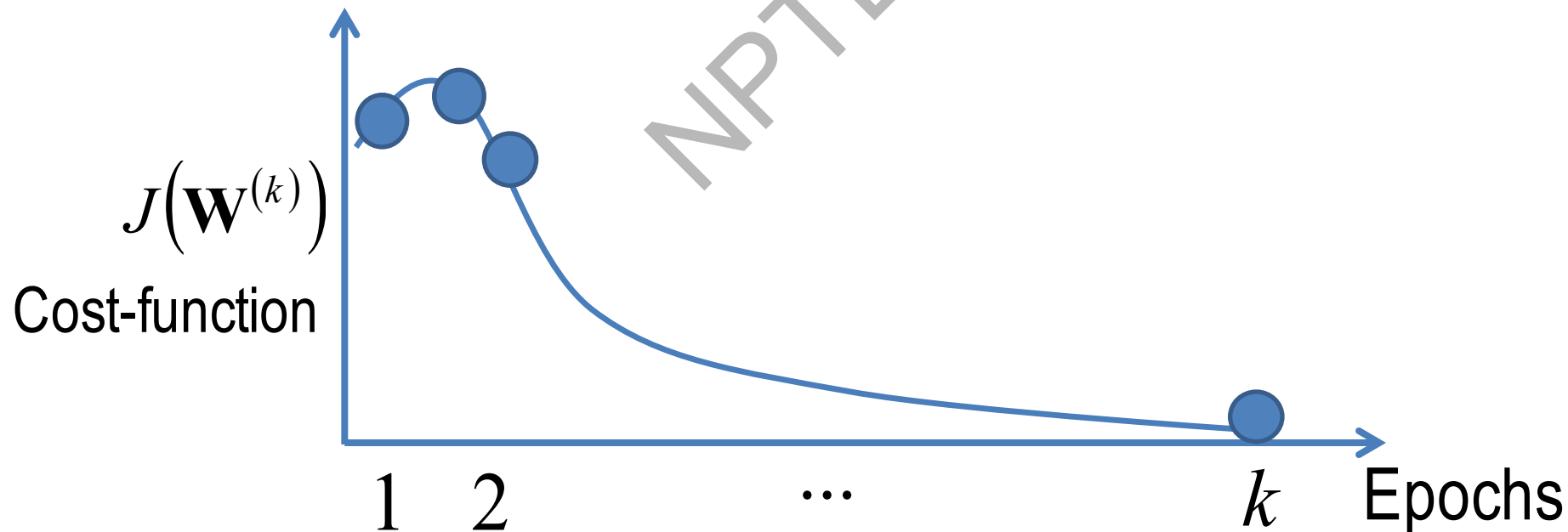
$$\mathbf{W} = \arg \min_{\mathbf{W}} \{J(\mathbf{W})\}$$

$$\mathbf{W}^{(k+1)} = \mathbf{W}^{(k)} - \frac{\partial}{\partial \mathbf{W}^{(k)}} J(\mathbf{W})$$



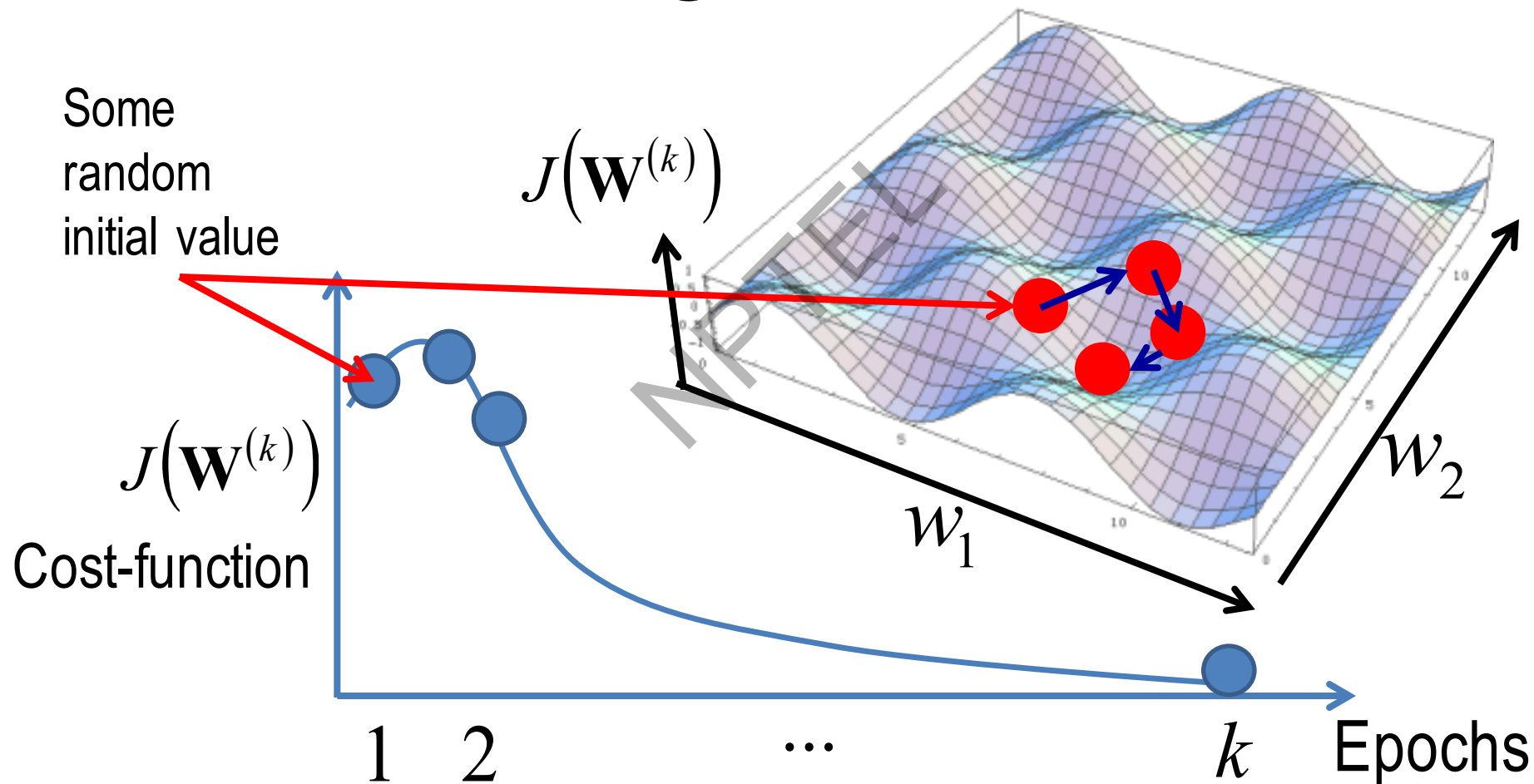
# Gradient Descent Learning

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \frac{\partial}{\partial \mathbf{w}^{(k)}} J(\mathbf{w})$$





# Understanding Gradient Descent





# Take Home Messages

- Haykin, Simon, *Neural Networks and Learning Machines*, 2001.
- Toolboxes
  - Matlab – Neural Network Toolbox (nprtool)
  - Python – Theano, scikits-learn
  - Lua – Torch, nn, cuDNN, nngraph